

RDC and HIT Internship Report

GPS Vehicle and Driver Tracking System

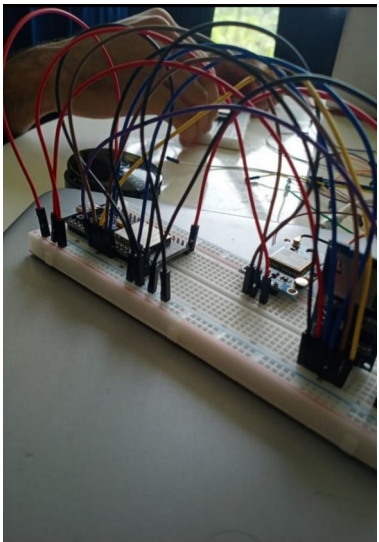
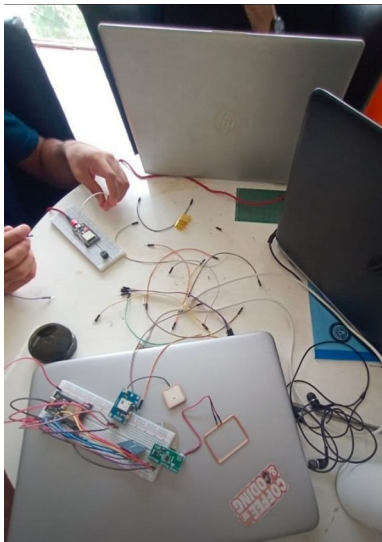
Embedded device integration, fleet monitoring and driver event analysis

Asad Hussain

Electrical Engineering
National University of Sciences and Technology
College of Electrical and Mechanical Engineering

Placement	Period
Research and Indigenous Development Centre College of E&ME	24 June to 8 August 2025
Heavy Industries Taxila Technical Directorate	7 July to 13 August 2025

Lead developer working within a project team



Prototype integration photographs supplied by the author. These show the bench development stage.

Revised 9 October 2026 | Personal technical report

Executive summary

This report presents my work as lead developer within a team on a GPS vehicle and driver tracking system during my RDC and HIT internships. The project connected an embedded logging device to MQTT communication, Python services, a SQL Server database and a browser dashboard. It addressed the need to associate vehicle journeys with drivers and give operators access to location history, maintenance records and driving events.

My technical contribution covered device integration, the trip logging workflow, backend processing and the operator interface. The supplied project files support a substantial implementation: separate ingestion and dashboard applications, live and historical telemetry handling, fleet administration, polygon geofencing and rule-based driving-event analysis. The original report records operational use; a dated deployment acceptance record was not included in the review package.

This revision checks the implementation against the evidence available. Fourteen isolated software checks were run during the revision: eleven met their stated expectations and three exposed validation or data-durability gaps. These checks do not replace hardware testing, database integration testing or field acceptance. Timing, capacity and zero-loss claims from the earlier report are excluded because supporting measurements were not supplied.

Report guide

Section	Page
Internship context and project objectives	3
System architecture	4
Hardware integration	5
Device workflow and operation	6
Backend and database design	7
Operator dashboard	8
Geospatial and driver event analysis	9
Software verification results	10
System acceptance plan	11
Project delivery and learning outcomes	12
Limitations and engineering improvements	13
Conclusion and references	14

Internship context and project objectives

The RDC certificate identifies the Research and Indigenous Development Centre at the College of E&ME and records work on designing and testing a driver logging device. The HIT certificate identifies the Technical Directorate. These placements are presented separately rather than treating RDC as a department of HIT. The HIT period is recorded as 7 July to 13 August 2025, as confirmed by me during this revision.

The project objective was to connect driver identity, vehicle telemetry and operational records in one workflow. The table maps each target to its deliverable and the available evidence. “Code reviewed” means a corresponding implementation exists; it does not mean an end-to-end acceptance test has passed.

Target	Deliverable and evidence	Status
Identify the driver	RFID login and logout; workflow document and prototype photos.	Documented
Capture journeys	GPS samples, local CSV storage and a documented one-second capture interval.	Firmware evidence pending
Transmit telemetry	MQTT callbacks for live data, batch sessions and device replies in server.py.	Code reviewed
Retain data offline	SD storage and retry procedure in the operating workflow.	Hardware test pending
Manage fleet records	Driver, vehicle, administrator and maintenance functions in app.py.	Code reviewed
Visualize journeys	Live tracking and route history templates with mapping interfaces.	Code and UI reviewed
Detect policy events	Polygon checks, speed thresholds and harsh-driving rules.	Code and selected logic tested
Support handover	Report, workflow, diagrams, source snapshot and an acceptance plan.	Documentation consolidated

Scope of this report

The scope is an embedded and software integration project. It does not claim certified automotive hardware, regulatory safety approval, measured fleet-wide savings or machine-learning-based driving classification. Event detection in the supplied source uses programmed rules.

System architecture

The device captures position and driver identity, keeps a local trip record and exchanges messages through an MQTT broker. Two distinct Python applications support the system: server.py receives and processes telemetry, while app.py serves the operator dashboard and its HTTP endpoints. SQL Server provides the shared persistence layer.

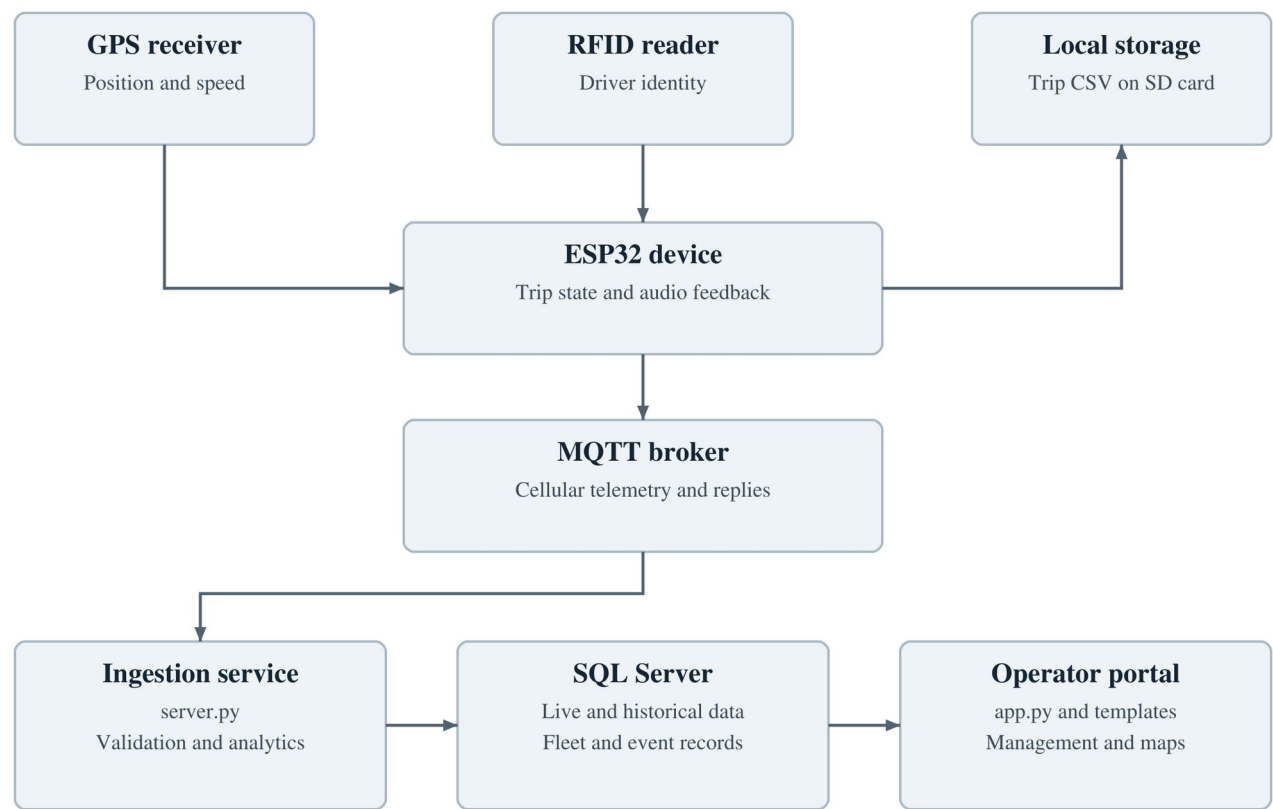


Figure 1. Functional architecture reconstructed from the supplied code and workflow. Arrows show principal data flow; acknowledgements and requests travel back through MQTT.

Layer	Technology	Purpose
Device	ESP32 platform, GPS, RFID and SD	Acquire trip data and provide local feedback.
Messaging	Cellular connection and MQTT	Carry live points, trip batches and replies.
Ingestion	Python, Flask, Paho and threading	Validate data and run event processing.
Storage	Microsoft SQL Server and pyodbc	Store fleet, telemetry and operational records.
Portal	Flask, Jinja2, HTML, CSS and JavaScript	Serve management pages and map views.

The broker, database and web services are logical components. The supplied material does not establish a final hosting topology or prove that a particular cloud deployment remained in service.

Hardware integration

The device brings together a microcontroller, a cellular connection, a GPS receiver, an RFID reader, removable storage and audible feedback. The photographs show a breadboard prototype and wiring work. They establish physical integration activity, while the original wiring illustration provides additional context for the module connections.

Component	Function	Evidence and qualification
LilyGO ESP32 cellular board	Device control and mobile data	Present in the design material; confirm the final board and modem part numbers.
GPS receiver and antenna	Position, time and speed	Present in the photographs and report; exact receiver model requires confirmation.
RDM6300 RFID reader	Read driver card identity	Identified in the wiring illustration and workflow.
SD storage	Driver cache and trip CSV files	Described in the operating workflow; firmware was not supplied.
TP4056 charging module	Battery charging arrangement	Shown in the illustration; battery and power path details need an as-built schematic.
Buzzer	Initialization and trip feedback	Shown in the design material and described in the workflow.
Breadboard and interconnects	Prototype assembly	Visible in the supplied bench photographs.

Integration approach

The embedded workflow coordinates initialization, driver identification, acquisition and transmission. GPS and RFID communication are described through serial interfaces, with SD storage supporting trip retention. The exact GPIO allocation, voltage domains and module configuration cannot be reconstructed reliably from photographs alone.

Hardware documentation needed for replication

The final hardware package should contain the firmware, a numbered bill of materials, an as-built schematic and pin map, supply ratings, battery specification and test records. The design should document modem current demand, stable supply behavior, antenna placement and connection strain relief before an enclosed vehicle installation is reproduced.

The earlier report names several GPS and modem variants. This revision uses functional component names where the sources disagree. It does not treat a wiring illustration as a verified electrical schematic or claim that battery charging automatically provides a validated power-sharing circuit.

Device workflow and operation

The sequence below follows System workflow.docx. It records intended device behavior; the embedded firmware was not part of the uploaded package, so its timing and failure handling have not been independently executed in this review.

Stage	Device behavior	Operator indication
1 Initialize	Mount SD storage; initialize modem and GPS; check mobile data.	One long beep on successful initialization; error beeps for module failures.
2 Connect	Attempt MQTT connection; request notifications when connected. Continue offline if the connection fails.	Three short beeps when ready for a card.
3 Identify	Present and remove the RFID card. Use cached driver data; fetch data when an unknown identity is scanned.	One confirmation beep after the scan.
4 Acquire	Start GPS capture after login and a valid fix. Documented target: one sample per second.	Two short beeps for the first GPS capture.
5 Retain and stream	Write trip data locally; documented flush interval is 30 seconds. Send live points while connected.	Trip remains active; avoid repeat card scans.
6 Close and upload	Scan again to end the trip. Send stored CSV batches and wait for server confirmation.	Two beeps for a confirmed file; three when the queue is complete.
7 Retry	Keep unconfirmed files for later transfer and recheck connectivity.	Return to the ready state after transfer completes.

Operator procedure

Power the device using the approved supply and wait for the ready signal. Present the card for approximately one to three seconds, remove it and allow time for confirmation. Wait for the GPS indication before moving. Keep the card away from the reader during the trip. Scan again at the end to close the trip and initiate the documented upload sequence.

Important operating boundaries

The supplied instructions require the server to be reachable and warn that a restart during a trip requires another login. The 30-second flush interval creates a potential window of unsaved data after abrupt power loss. Retention, restart recovery and the relationship between confirmation and durable storage require the acceptance tests in this report.

One source says driver data is fetched at boot, while the workflow says boot fetching is skipped and unknown cards trigger retrieval. The uploaded server contains a boot-response path. The final firmware and message contract are needed to confirm which sequence was deployed.

Backend and database design

The ingestion application receives JSON messages, identifies the device and driver, buffers batch sessions and handles live points. Live telemetry is written to GPSTData_live. Historical trip processing uses GPSTData and invokes mileage, driving-event and geofence routines. Background threads move batch work out of the immediate message path.

The dashboard application separately exposes fleet management, maintenance, notifications, live tracking and route history functions. The source snapshot contains 90 Flask route declarations in app.py and six in server.py. These are source counts, not counts of independently tested or distinct API capabilities.

Record group	Tables referenced by the code	Role
Fleet and identity	vehicles, drivers, admins	Vehicle details, driver records and operator accounts.
Live telemetry	GPSTData_live	Current or recent device positions.
Trip history	GPSTData	Historical coordinates, timestamps and speed.
Driving events	events	Recorded speeding and harsh-driving events.
Geofences	zones, zone_points, zones_for_vehicle	Polygons and vehicle-to-zone assignments.
Operator alerts	notifications	Geofence and maintenance notifications.
Maintenance	maintenance_records	Service records, periodicity and due thresholds.

Logical persistence model inferred from SQL queries. The database creation scripts, constraints and indexes were not supplied.

Data handling

The live validator checks required fields, numeric coordinates, coordinate bounds and negative speed. Duplicate checks compare vehicle, driver and timestamp before insertion. The batch path verifies device and driver records and processes points in a background worker. Query-level duplicate checks do not establish database-level uniqueness under concurrent writes.

Storage acknowledgement

The supplied implementation can report a successful batch receipt after starting a worker and before its database commit. Accordingly, a device confirmation currently means initial acceptance for processing. A stronger design would commit a durable batch record before authorizing local deletion and use a stable trip identifier to make replays idempotent. This improvement is proposed, not applied to the uploaded source.

Operator dashboard

The portal provides driver and vehicle management, maintenance scheduling, live and historical tracking, administrator management, notifications and geofence configuration. The archive contains ten functional page templates and two error templates. The dashboard groups its primary functions into six navigation cards.

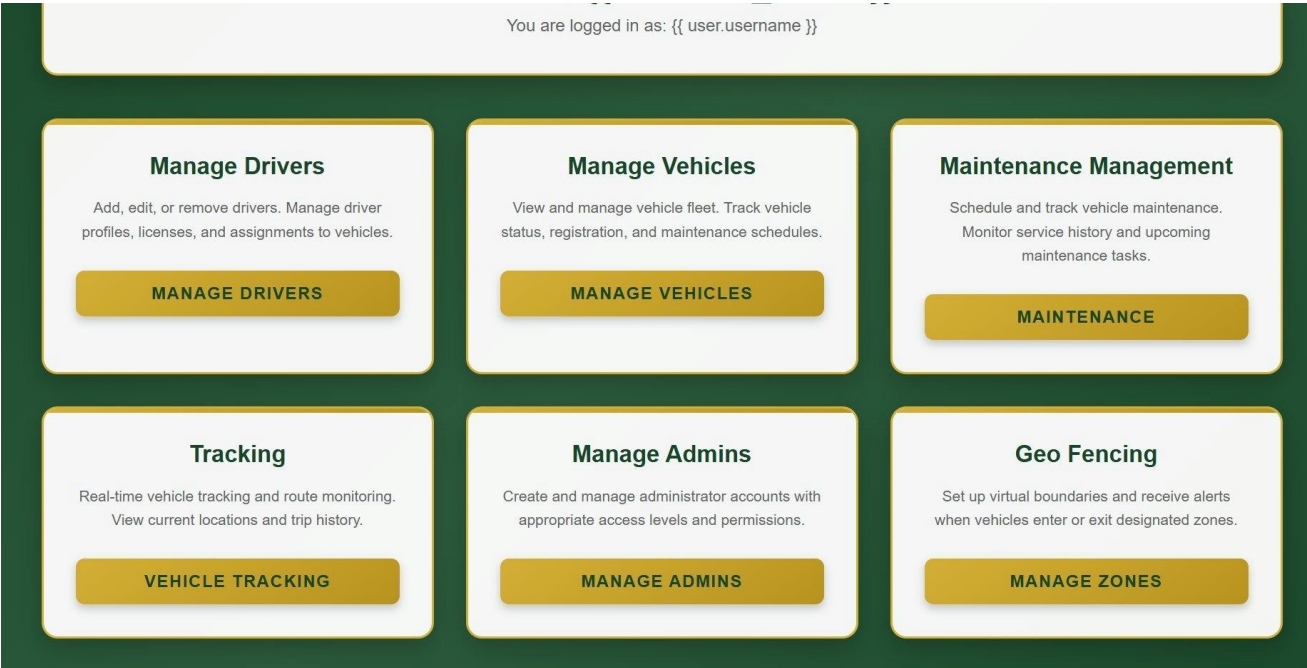


Figure 2. Dashboard image from the original report. The unresolved username placeholder is visible in the supplied capture; this image is interface evidence, not proof of an authenticated live session.

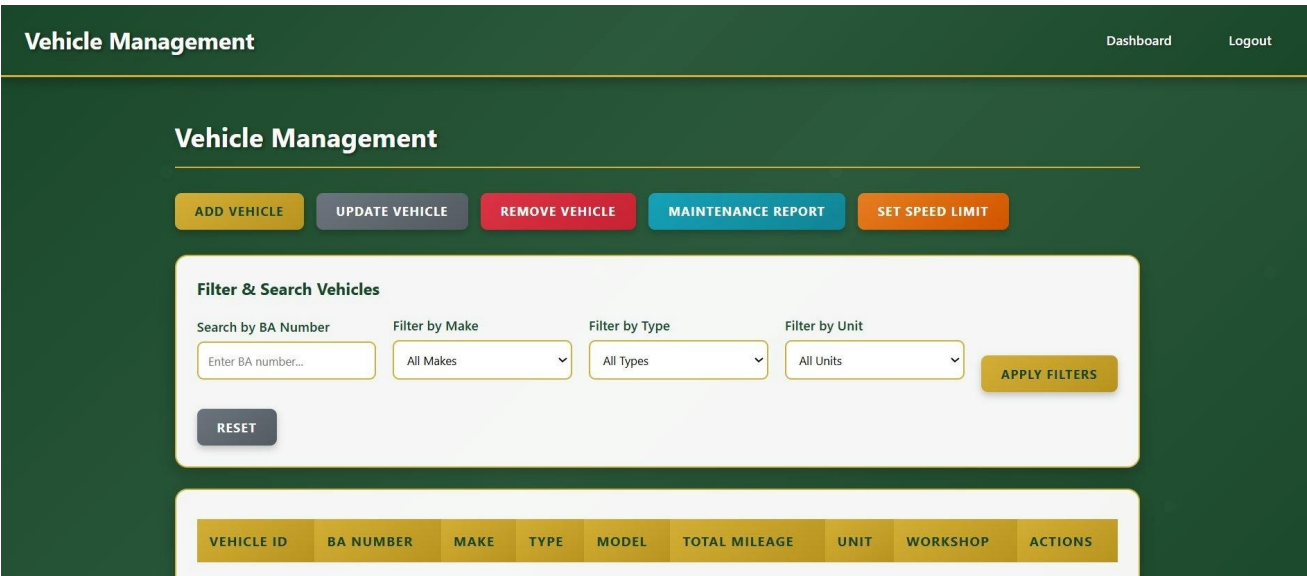


Figure 3. Vehicle management interface from the original report. No operational vehicle records are reproduced here.

The source includes vehicle filters, driver and vehicle reports, CSV export, service criteria, unread notifications and map-related endpoints. Maintenance checks use dates, mileage thresholds or both. The route and live-tracking templates use mapping interfaces to distinguish current position from historical journeys.

Geospatial and driver event analysis

Polygon geofencing

The server uses a ray-casting test to determine whether a point lies inside a polygon. It compares the containing zone with the zones assigned to a vehicle. In the supplied code, an alert is generated on entry into a defined zone that is not assigned to that vehicle. This is narrower than a universal alert whenever a vehicle leaves every permitted area.

The transition state is recreated for each batch and the first matching polygon determines the result. Overlapping zones, points on boundaries and continuity between batches therefore need explicit acceptance cases. The isolated review tests confirmed one interior and one exterior point for a simple square.

Harsh driving rules

The algorithm compares speed changes across adjacent samples and also looks ahead by two to four points. Thresholds depend on the time interval. These are heuristic speed-change rules, not a trained model or a calibrated accelerometer measurement.

Time between adjacent points	Acceleration threshold	Braking threshold
More than 0 and up to 3 seconds	+25 km/h	-25 km/h
More than 3 and up to 6 seconds	+35 km/h	-35 km/h
More than 6 and up to 10 seconds	+40 km/h	-30 km/h
More than 10 and up to 60 seconds	+50 km/h	-40 km/h

Adjacent-point thresholds in the supplied server. Look-ahead comparisons use a separate set of thresholds.

Road categories and speeding

The server queries OpenStreetMap through Overpass, caches a category by rounded coordinates and compares speed with a vehicle-specific threshold. It selects the first nearby road returned rather than proving a match to the travelled road. In the supplied mapping function, the expressway condition cannot be satisfied by the listed trunk tag values, so that category requires correction and testing.

Distance and measurement limits

Distance is calculated by summing geodesic segments between successive points. GPS noise, missing samples and irregular timestamps can affect distance and event detection. Coordinate rounding expresses stored precision, not receiver accuracy. No labelled driving dataset was supplied to establish false-positive or false-negative rates.

Software verification results

During the 9 October 2026 report revision, selected functions were extracted from the supplied server.py and executed with synthetic inputs. Database access, event insertion and thread execution were replaced with controlled stubs. The application was not started and no broker, operational database or device was contacted.

Check	Observed result	Outcome
Valid payload	Accepted required fields and valid numeric values.	Pass
Missing driver field	Rejected the payload.	Pass
Latitude outside bounds	Rejected latitude 91.	Pass
Negative speed	Rejected speed below zero.	Pass
Malformed timestamp	Accepted a non-date timestamp at validation.	Gap
Infinite speed	Accepted positive infinity at validation.	Gap
Interior polygon point	Returned inside for a square.	Pass
Exterior polygon point	Returned outside for a square.	Pass
Motorway mapping	Mapped motorway to Motorway.	Pass
Trunk mapping	Mapped trunk to National Highways.	Pass
Constant speed	Created no harsh-driving event.	Pass
Rapid speed increase	Classified a 0 to 30 km/h change in one second.	Pass
Rapid speed decrease	Classified a 35 to 0 km/h change in one second.	Pass
Confirmation after storage	Returned success while the worker had not run.	Gap

Eleven of fourteen checks met the stated expectations. The three gaps show that input validation and storage confirmation need strengthening. Passing the motorway and trunk checks confirms the implemented mapping behavior; it does not validate road classification against ground truth.

These results concern small, isolated software paths in the uploaded snapshot. They are not internship-era field measurements, an overall reliability score or evidence of complete system readiness.

System acceptance plan

The following tests define the evidence needed to close the main project targets. They are proposed acceptance checks and were not executed during this report revision. Quantitative limits should be agreed with the supervisor and operating team before testing.

Scenario	Acceptance evidence	Record to retain
Boot and module failure	Device reaches ready state only when required modules initialize; each failure gives the defined indication.	Firmware revision and a test video or log.
RFID lifecycle	Valid cards start and end the correct trip; unknown cards and repeated scans follow the agreed policy.	Card test matrix using anonymous IDs.
GPS acquisition	A valid fix starts logging; missing or stale fixes are identified. Measure actual sample intervals.	Timestamped acquisition log.
Live telemetry	Device, broker, database and dashboard records reconcile for a known point sequence.	Expected and stored counts plus latency samples.
Network outage	Local records survive a defined disconnection and upload after recovery.	Before-and-after files and database counts.
Restart and power loss	Recovery behavior and any loss before a storage flush are measured.	Restart sequence and recovered record counts.
Retry and duplicates	Repeating the same trip does not double telemetry, mileage or events.	Stable trip IDs and database reconciliation.
Database failure	No durable-success reply is issued before storage succeeds; failed batches remain recoverable.	Failure injection and retry logs.
Geofence boundaries	Inside, outside, boundary, overlap and cross-batch cases follow the approved policy.	Labelled synthetic routes and expected alerts.
Driving events	Threshold boundaries and noisy GPS are compared against labelled expected events.	Precision and recall with dataset description.
Operator access	Login, authorization, session expiry and state-changing requests follow the access policy.	Role and endpoint test results.
Endurance and handover	Representative load, outage recovery, backup restoration and operator use are demonstrated.	Test duration, fleet size and signed acceptance.

Project delivery and learning outcomes

My role and contribution

I worked as lead developer within a project team. My contribution connected the device workflow with backend processing and the operator interface. The work required translating tracking and driver-identification needs into hardware interfaces, message formats, data records and visible application behavior. This report does not claim sole authorship of every component or unverified authority over budgets, staffing or organizational decisions.

Delivery sequence

The deliverables can be organized into five engineering stages: requirements and device workflow; hardware integration; telemetry and persistence; dashboard and analytics; and testing and handover. This sequence explains how the work fits together. It is not a reconstructed weekly attendance log because dated task records were not supplied.

Stage	Output	Dependency or review point
Define	Workflow and functional targets	Agree trip boundaries, driver policy and operator needs.
Integrate	Bench prototype and interfaces	Confirm components, power and serial communication.
Connect	MQTT and database paths	Align message fields and storage semantics.
Present	Management pages and analytics	Validate records and rules with representative data.
Handover	Report and operating procedure	Complete acceptance evidence and ownership transfer.

Skills developed

The project strengthened my experience in embedded integration, serial peripherals, state-based device behavior, MQTT messaging, Python backend development and relational data handling. Implementing geospatial calculations and time-series rules required attention to units, sample timing and edge cases. Connecting these components also improved my debugging and technical documentation practices.

Engineering lessons

A system needs a shared definition of success at each boundary. A broker delivery event, a server receipt and a committed database record are different milestones. Local buffering is useful only when deletion and replay rules preserve that distinction. Similarly, an event marker on a map needs a documented rule and a labelled test case before its accuracy can be assessed.

The original report records deployment and practical use. This revision presents those as reported outcomes, while the uploaded source, diagrams and photographs directly support the implementation work. Fleet size, uptime, operating cost reductions and other measured business outcomes remain unquantified.

Limitations and engineering improvements

The recommendations below follow from the supplied snapshot and the review checks. They are future work; this report revision does not modify the operational application or assert that these changes have already been deployed.

Priority	Improvement	Completion evidence
1	Persist each batch before sending a durable-success acknowledgement. Add stable trip IDs, replay protection and recoverable work queues.	Failure and restart tests reconcile all accepted records without double-counted mileage.
1	Validate finite numbers, timestamp format and permitted identities consistently on live and historical paths.	Malformed and unauthorized payloads are rejected with recorded reasons.
1	Review operator authentication and authorization. Replace plain-password comparison with consistent password-hash verification and secure session configuration.	Access tests and password migration review pass.
1	Move configuration out of source; use authenticated, encrypted connections and scoped device permissions.	Configuration review confirms protected secrets and restricted access.
2	Make subscription QoS and session persistence explicit. The source uses <code>clean_session=True</code> and subscriptions without an explicit QoS value.	Broker and device tests document negotiated behavior and recovery.
2	Correct road classification and unify analytics shared between the ingestion and dashboard services.	Labelled road and event cases produce consistent results.
2	Define geofence boundary, overlap and cross-batch behavior; add database uniqueness and transactional updates.	Boundary, replay and concurrent-write tests pass.
3	Complete the as-built hardware package, logging, monitoring, dependency versions, backups and a deployment runbook.	Another engineer reproduces the system and completes handover tests.

The earlier report described universal QoS 1 delivery, persistent sessions, sub-50 ms callbacks, zero data loss and zero false positives. The supplied code and test evidence do not establish those claims. They should be replaced with measured values only after repeatable tests record the environment, dataset, load and failure conditions.

Conclusion and references

The RDC and HIT project demonstrates my contribution to an integrated embedded and software system for driver logging and fleet monitoring. The implementation connects physical sensing, cellular messaging, backend processing, relational storage and operator-facing tools. The revised report makes the project targets traceable to artifacts and identifies the remaining evidence needed for dependable handover.

The strongest next step is to complete the firmware and database package, correct the identified durability and validation issues, and run the acceptance matrix with the project team. That would turn the current implementation evidence into a reproducible and measurable system record.

Project sources

- [1] Asad Hussain. Internship_Report.pdf. Supplied 13-page report. Used for original project scope, photographs of interfaces and reported deployment context.
- [2] server.py. Uploaded telemetry ingestion source, 1,191 lines. Reviewed without executing its service startup or contacting configured infrastructure.
- [3] dld_app.zip. Dashboard source archive. Review focused on app.py, twelve HTML templates, requirements.txt and Procfile; the bundled virtual environment was not treated as project evidence.
- [4] System workflow.docx and block diagram.docx. Supplied device sequence, operator instructions and module wiring illustration.
- [5] Two original prototype photographs supplied on 9 October 2026. Used to document the bench integration stage.
- [6] RDC and HIT internship certificates in the embedded portfolio. RDC dates: 24 June to 8 August 2025. HIT dates: 7 July to 13 August 2025, with the year confirmed by the author during this revision.

Technical references

- [7] Eclipse Paho MQTT Python client documentation. Session behavior, publish and subscribe parameters. Accessed 9 October 2026.
<https://eclipse.dev/paho/files/paho.mqtt.python/html/client.html>
- [8] Werkzeug security utilities. Password-hash generation and verification. Accessed 9 October 2026.
<https://werkzeug.palletsprojects.com/en/stable/utis/>

Supplementary evidence for formal submission

Add the supervisor name and designation, university registration number, final firmware, SQL schema, dated test logs and deployment acceptance record if the receiving institution requires them. A weekly log should be added only from actual attendance or task records.

This public report excludes service credentials, server addresses, personal driver records and operational route data. It is authored by the intern and does not serve as an institutional endorsement or acceptance certificate.